

-: Documentation File :-

Penny Stock Predictor

Introduction:-

An AI-powered web application that predicts stock price trends using **Long Short-Term Memory (LSTM) neural networks**.

The system leverages **historical financial data from Yahoo Finance**, applies **time-series preprocessing techniques**, and forecasts the **next day's stock price**, trend direction, and expected accuracy.

Features:-

 Fetches **real-time stock market data** from Yahoo Finance.

 Implements **LSTM deep learning model** optimized for time-series forecasting.

 Predicts:

- Actual vs Predicted stock prices.
- Next day's closing price.
- Trend direction (**up / down**).
- Accuracy estimation (%).

 Supports **.keras** and **.h5** model formats for flexibility.

 Auto-trains on first run if no saved model is found.

 Provides **REST API** for integration with web or mobile apps.

Tech Stack:-

Programming Language: Python

Backend Framework: Flask (with Flask-CORS for cross-origin requests)

Machine Learning:

- TensorFlow / Keras (LSTM model)

- Scikit-learn (MinMaxScaler)

Data Source: Yahoo Finance (yfinance API)

Data Handling: NumPy, Pandas

Model Persistence: Joblib (for scaler), TensorFlow SavedModels

Frontend: HTML, CSS, Chart.js (for visualization)

Installation & Setup:-

1. Prerequisites

- Python 3.8+
- pip (Python package manager)

```
# Clone the repository
git clone https://github.com/yourusername/penny-stock-predictor.git
cd penny-stock-predictor

# Create virtual environment
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# Install dependencies
pip install -r requirements.txt

# Run the Flask app
python app.py
```



The server will start at:

👉 <http://127.0.0.1:5000/>

Usage:-

Web Interface

1. Navigate to <http://127.0.0.1:5000/>.
2. Enter a stock ticker symbol (e.g., AAPL, TSLA, POWERGRID.NS).

3. View predictions and accuracy.

API Endpoint:-

POST /predict

Request (JSON):-

```
json
{
  "symbol": "AAPL"
}
```

Response (JSON):-

```
{
  "symbol": "AAPL",
  "dates": ["2025-08-20", "2025-08-21", ...],
  "actual": [186.45, 187.12, ...],
  "predicted": [186.78, 187.89, ...],
  "current_price": 187.34,
  "next_day_prediction": 189.45,
  "price_change_percent": 1.12,
  "trend": "up",
  "accuracy": 96.78,
  "model_status": "trained"
}
```

Methodology:-

The **prediction pipeline** follows a structured methodology:

1. Data Collection

- Historical stock data (5 years by default) is retrieved from **Yahoo Finance API**.
- Features considered: **Closing Price** (focus for forecasting).

2. Data Preprocessing

- Missing values are handled.
- **MinMaxScaler** scales data to the [0,1] range for stable neural network training.
- The dataset is split into:
 - **Training set** (70–80%)
 - **Testing set** (20–30%)
- Sliding window approach: last **100 days of stock prices** → used to predict the next day.

3. Model Architecture (LSTM)

- **Layer 1:** LSTM (50 units, ReLU, return sequences) + Dropout (0.2).
- **Layer 2:** LSTM (60 units, return sequences) + Dropout (0.3).
- **Layer 3:** LSTM (80 units, return sequences) + Dropout (0.4).
- **Layer 4:** LSTM (120 units, final sequence) + Dropout (0.5).
- **Output Layer:** Dense(1) → predicted closing price.
- **Optimizer:** Adam
- **Loss Function:** Mean Squared Error (MSE)

4. Training Process

- Epochs: **10**
- Batch size: **32**
- Validation split: **10%**
- Model is saved as .keras or .h5 for reuse.
- Scaler is saved separately using **Joblib**.

5. Prediction Process

- Last **100 days of prices** are extracted and scaled.
- Model generates the **next day's predicted price**.
- Results are rescaled back to original price values.

- RMSE (Root Mean Squared Error) is calculated as accuracy metric.

6. Output Metrics

- Actual vs Predicted price series.
- Next day's predicted price.
- % change in price.
- Trend direction (↑ / ↓).
- Accuracy percentage.

Model Explanation:-

- **Why LSTM?**
LSTM networks are specifically designed for **time-series forecasting** because they capture **long-term dependencies** better than traditional RNNs.
- **Training Data Strategy**
Only **closing prices** are considered, ensuring simplicity while maintaining predictive power.
- **Evaluation Metric**
RMSE is used since it penalizes larger errors more strongly, suitable for stock price prediction.

Future Improvements:-

- Add **multivariate features** (Open, High, Low, Volume).
- Use **Transformer/Attention-based models** for better long-term forecasting.
- Integrate **sentiment analysis** (Twitter, news, Reddit).
- Deploy as a **cloud API** (AWS/GCP/Heroku).
- Add **interactive dashboards** using Plotly Dash or Streamlit.

Conclusion:-

The **Penny Stock Predictor** successfully demonstrates the application of **deep learning techniques (LSTMs)** in financial time-series forecasting.

By leveraging historical market data, preprocessing methods, and sequential neural networks, the system provides **reasonably accurate short-term predictions** of stock prices and trends.

While financial markets are influenced by numerous unpredictable factors such as **economic policies, geopolitical events, and trader psychology**, this project proves that machine learning can serve as a **powerful decision-support tool**.

With further enhancements—like incorporating **multivariate features, sentiment analysis, and advanced models**—the system has the potential to evolve into a robust financial analytics platform for investors and researchers alike.